

Computer Graphics

Week 9: Texture Mapping



Texture Coordinates

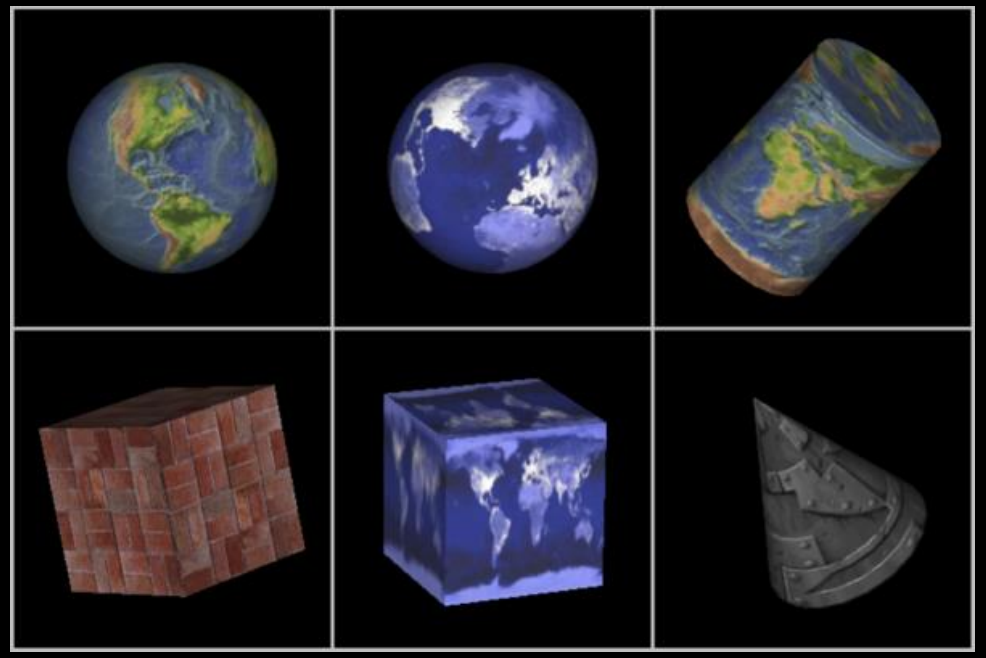
The Limits of Geometric Modeling

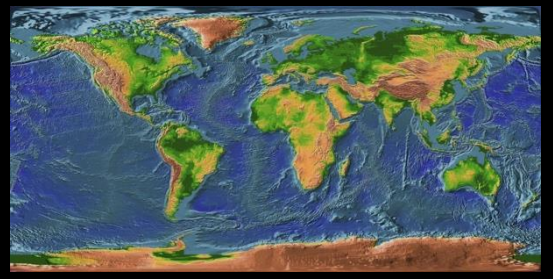
- Although graphics cards can render over 10 million polygons per second, that number is insufficient for many phenomena
  - Clouds
  - Grass
  - Terrain
  - Skin

The Limits of Uniform Colors

- Three-dimensional objects can be made to look more interesting and more realistic by adding a texture to their surfaces
- Texture: sort of variation from pixel to pixel within a single primitive
- Image texture: can be applied to a surface to make the color of the surface vary from point to point, something like painting a copy of the image onto the surface

Sample Image Textures and 3D Objects Rendered with Textures







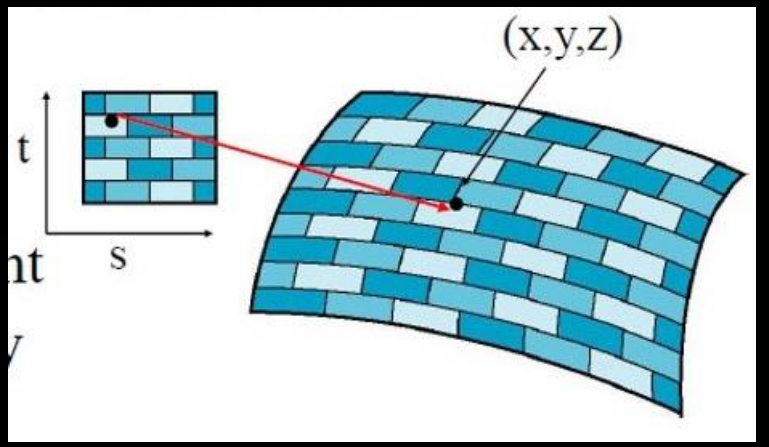




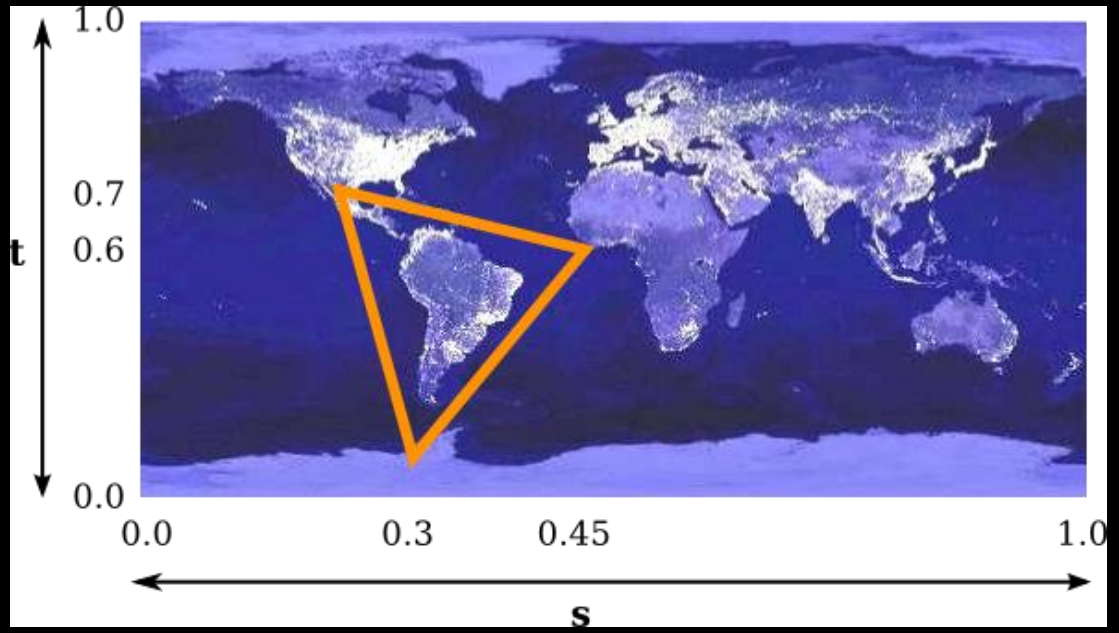
Texture Coordinates

- When a texture is applied to a surface, each point on the surface must correspond to a point in the texture
- For that, the object needs texture coordinates
- Texture coordinates are specified for each vertex of a primitive
- Texture coordinates for points inside the primitive are calculated by interpolating the values from the vertices of the primitive

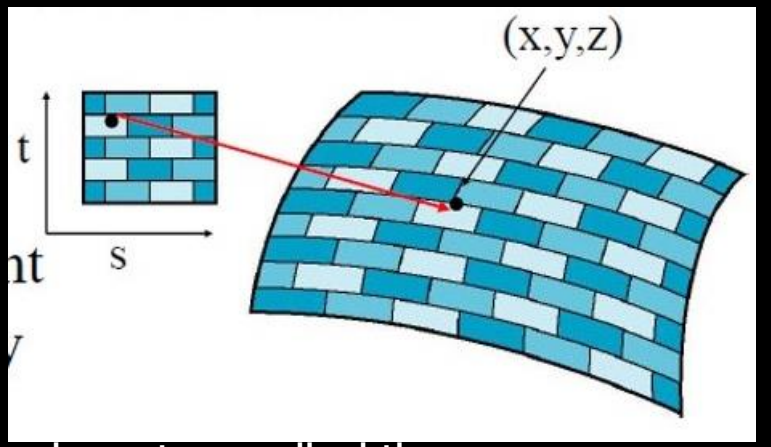
Texture Coordinates



- Texture image is in a 2D coordinate system
  - *s* is used for the horizontal coordinate: 0 – 1 from left to right of the image
  - *t* is used for the vertical coordinate: 0 – 1 from bottom to top of the image



Texture Coordinates



- To draw a textured primitive, we need a pair of numbers (s,t) for each vertex, called the texture coordinates for that vertex
- Texture coordinates are typically provided to the shader program as an attribute variable
- Generated texture coordinates should usually be computed from the object coordinates of the object that is being rendered: they are computed from the original vertex coordinates, before any transformation has been applied
- When the object is transformed, the texture will be transformed along with the object so that it will look like the texture is attached to the object

Texture Coordinates

- The simplest idea for generated texture coordinates is simply to use the x and y coordinates from the object coordinate system as the texture coordinates

- If the vertex coordinates are given as the value of the attribute variable `a_coords`, that would mean using `a_coords.xy` as texture coordinates

- The mapping works OK for a polygon that is facing in the direction of positive z, but it doesn't give good results for polygons that are edge-on to the xy-plane



Texture Coordinates

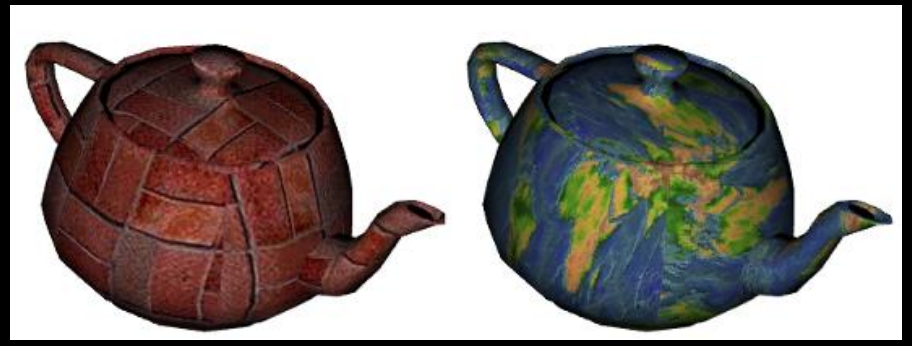
- The texture projects nicely onto the front face of the cube
- It also works OK on the back face of the cube (not visible in the image), except that it is mirror-reversed
- On the other four faces, which are exactly edge-on to the xy-plane, you just get lines of color that come from pixels along the border of the texture image

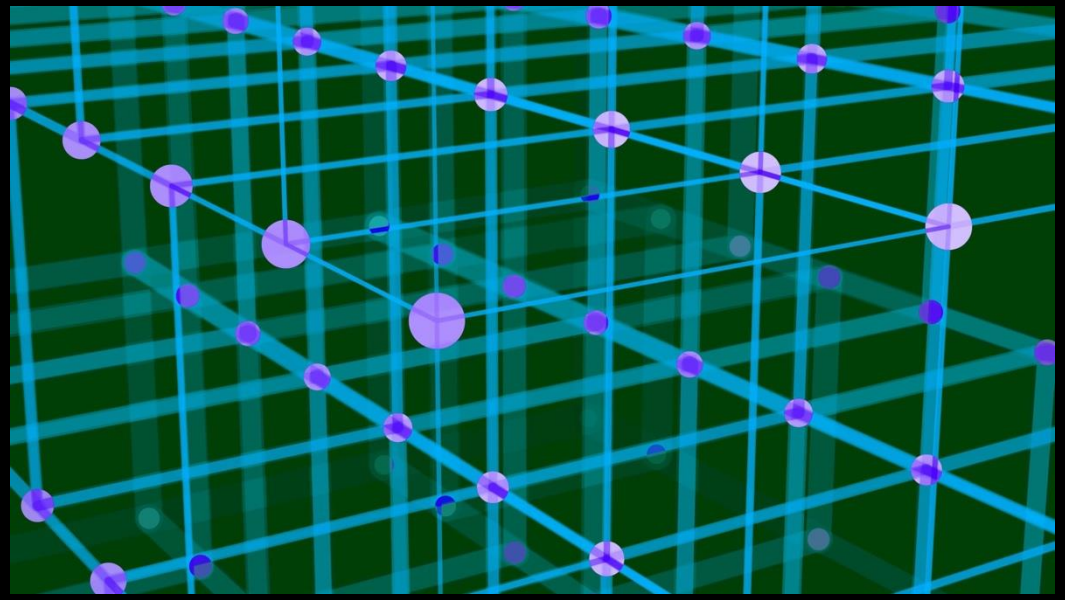


Texture Coordinates

- We could project in other directions to map the texture to other faces of the cube
- Let's say that we want to project along the direction of one of the coordinate axes
- The normal vector to the surface tells us that direction
- We should project in the direction where the normal vector has its greatest magnitude

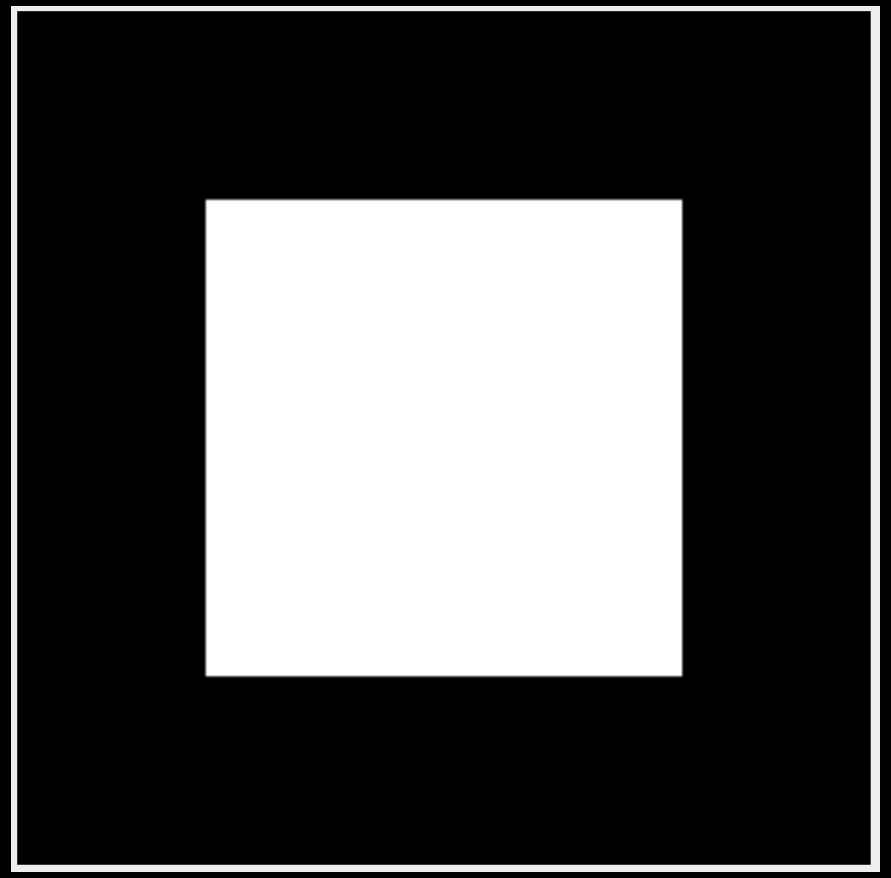
- Which axes to use if the normal vector is (0.12, 0.85, 0.51)?
- (−0.4, 0.56, −0.72)?





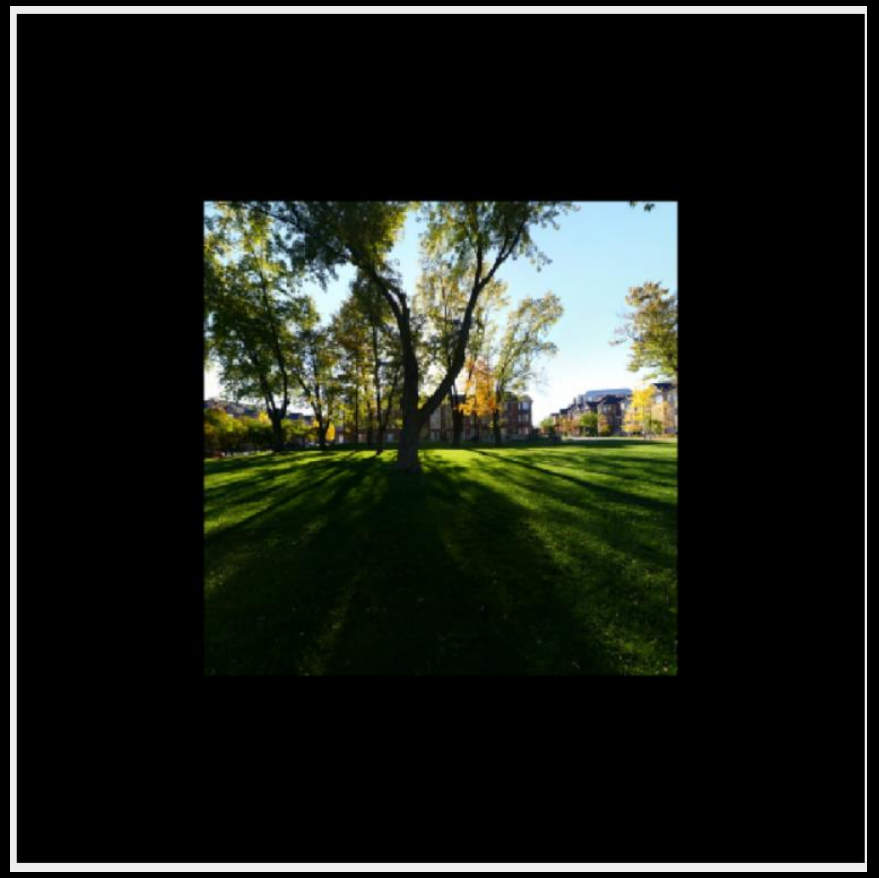
Texture Mapping Using WebGL

From this



- the front face of a white 2-by-2 cube that can be rotated using the keyboard

To this



- The textured surface of a cube that can be rotated using the keyboard

Need this



- The image texture

The Vertex Shader

```
attribute vec2 a_texCoords;

varying vec2 v_texCoords;

varying vec4 v_color;
```

- In the `main` function

```
vec3 N = normalize( u_normalMatrix * u_normal );
v_color = vec4( abs(N.z) * u_diffuse, 1.0 );
v_texCoords = a_texCoords;
```

The Fragment Shader

```
varying vec4 v_color;
varying vec2 v_texCoords;
uniform sampler2D u_sampler;
```

- In the `main` function

```
vec4 texColor = texture2D( u_sampler, v_texCoords);
gl_FragColor = v_color * texColor;
```




```
let textureObjects = [];  // Six texture objects to hold the six tectures.

const textureURLs = [
      "park/posz.jpg",
      "park/posx.jpg",
      "park/negz.jpg",
      "park/negx.jpg",
      "park/posy.jpg",
      "park/negy.jpg"
];
```

The initGL function

- The 3D object: a square (front face of a cube)

```
let aCoordsLoc = gl.getAttribLocation(prog, "a_coords");
let coordsBuf = gl.createBuffer();
gl.bindBuffer( gl.ARRAY_BUFFER, coordsBuf );
let coords = new Float32Array( [ -1,-1,1, 1,-1,1, 1,1,1, -1,1,1 ] );
gl.bufferData( gl.ARRAY_BUFFER, coords, gl.STATIC_DRAW );
gl.vertexAttribPointer(aCoordsLoc, 3, gl.FLOAT, false, 0, 0);
gl.enableVertexAttribArray(aCoordsLoc);
```

The initGL function

- The Image Texture Coordinates

```
let aTexCoordsLoc = gl.getAttribLocation(prog, "a_texCoords");
let tCoordsBuf = gl.createBuffer();
gl.bindBuffer( gl.ARRAY_BUFFER, tCoordsBuf );
let tCoords = new Float32Array( [ 0,0, 1,0, 1,1, 0,1 ] );
gl.bufferData( gl.ARRAY_BUFFER, tCoords, gl.STATIC_DRAW );
gl.vertexAttribPointer(aTexCoordsLoc, 2, gl.FLOAT, false, 0, 0);
gl.enableVertexAttribArray(aTexCoordsLoc);
```

- Flip how the webGL reads texture coordinates:

```
gl.pixelStorei(gl.UNPACK_FLIP_Y_WEBGL,1);
```

The initGL function

- The Normal Vector and Diffuse Color

```
uTransformMatrixLoc = gl.getUniformLocation(prog, "u_transformMatrix");
uNormalMatrixLoc = gl.getUniformLocation(prog, "u_normalMatrix");

let uNormalLoc = gl.getUniformLocation(prog, "u_normal");
gl.uniform3f(uNormalLoc, 0, 0, 1);
let uDiffuseLoc = gl.getUniformLocation(prog, "u_diffuse");
gl.uniform3f(uDiffuseLoc, 1, 1, 1);
```

The initGL function



- Loading Texture

```
let img = new Image();
img.onload = function() {
    textureObjects[0] = gl.createTexture();
    gl.bindTexture(gl.TEXTURE_2D, textureObjects[0]);
    gl.texImage2D(gl.TEXTURE_2D, 0, gl.RGBA, gl.RGBA, gl.UNSIGNED_BYTE, img);
    gl.generateMipmap(gl.TEXTURE_2D);
    console.log("loaded texture", 0);
    draw();
};
img.src = textureURLs[0];
```

The draw function

```
function draw() {
    gl.clearColor(0,0,0,1);
    gl.clear(gl.COLOR_BUFFER_BIT | gl.DEPTH_BUFFER_BIT);

    /* Set up modelview to hold the viewing transform. */

    mat4.lookAt(modelview, [0,0,10], [0,0,0], [0,1,0]);
    mat4.rotateX(modelview, modelview, rotateX);
    mat4.rotateY(modelview, modelview, rotateY);
    mat4.rotateZ(modelview, modelview, rotateZ);

    // Save the viewing transformation.
    let saveMV = mat4.clone(modelview);

    drawSquare(textureObjects[0]);  // front face

}
```

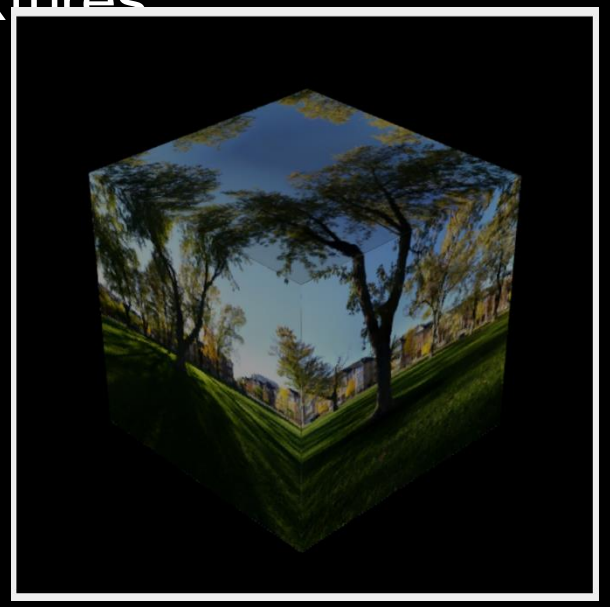
The drawSquare function

```
function drawSquare(texObj) {
    // Send combined projection/modelview matrix to shader.
    mat4.multiply( modelviewProj, projection, modelview );
    gl.uniformMatrix4fv(uTransformMatrixLoc, false, modelviewProj );
    // Send normal matrix to the shader.
    mat3.normalFromMat4(normalMatrix, modelview);
    gl.uniformMatrix3fv(uNormalMatrixLoc, false, normalMatrix);
    // bind the appropriate texture object
    gl.bindTexture(gl.TEXTURE_2D,texObj);
    // Draw the 2-by-2 square (originally in the xy-plane)
    gl.drawArrays(gl.TRIANGLE_FAN,0,4);
}
```

Sample Texture Mapping using WebGL

- Continue the sample program so that it can show the 6 sides of a cube using texture mapping and the provided image textures







Next Agenda

Students demonstrate their program